2026-9-18 Linux网络--TCP协议基础与报文

1. TCP 协议基础与报文

TCP 是面向连接、面向字节流的可靠传输协议。本篇只负责回答 TCP 的“数据模型”和“报头语义”：应用数据如何进入协议栈、TCP 与 UDP 有什么不同、TCP 报头中的字段分别负责什么，以及应用消息边界为什么必须自己划分。

连接建立、确认重传、流量控制、拥塞控制和 Socket 编程分别由后续专题展开。阅读顺序建议为：

1. 本篇：先理解报文和字节流；
2. 2026-9-18 Linux网络--TCP可靠传输与流量控制：理解 ACK、序号和窗口；
3. 2026-9-18 Linux网络--TCP拥塞控制与传输优化：理解网络拥塞与效率；
4. 2026-9-18 Linux网络--TCP连接管理与异常诊断：理解握手、挥手和状态；
5. 2026-9-18 Linux网络--TCP Socket编程与工程实践：理解 API、服务器和部署。

2. 传输层的通信模型

2.1 从主机通信到进程通信

2.1.1 IP 与端口的分工

IP 层负责把数据包从一台主机跨网络送到另一台主机；传输层还要决定数据应该交给目标主机上的哪个进程。

```
目的 IP 地址 -> 目标主机
目的端口号   -> 目标主机上的具体网络进程
源 IP、源端口 -> 标识发送方并支持回包
```

TCP 和 UDP 都使用 16 位端口号。端口号不是应用协议的消息编号，也不能区分同一连接内的不同业务消息；它只负责 socket 和进程的定位。

2.1.2 报头与有效载荷的分界

接收端处理传输层报文时，必须回答两个问题：

1. 哪一段是协议报头，哪一段是上层有效载荷？

2. 有效载荷应该交给哪个上层协议或进程?

TCP 使用四位首部长度确定报头边界，用目的端口完成向上分用；UDP 使用固定 8 字节报头和长度字段确定数据报边界，再用目的端口分用。

2.2 UDP 数据报模型

2.2.1 无连接与面向数据报

UDP 是用户数据报协议。它不需要三次握手，也不维护类似 TCP 的连接状态。服务端把地址绑定到 `SOCK_DGRAM` 套接字后，可以不断调用 `recvfrom` 接收不同客户端的数据报，再通过 `sendto` 回复。

UDP 每次发送形成一个独立数据报，接收端按数据报边界读取。因此 UDP 不存在 TCP 字节流意义上的粘包，但协议本身不保证到达、去重和顺序。

2.2.2 固定 8 字节报头

字段	长度	作用
源端口	16 位	标识发送方进程，可回包时使用
目的端口	16 位	接收端据此完成分用
UDP 长度	16 位	报头与有效载荷的总长度
UDP 校验和	16 位	检测传输错误

固定报头使 UDP 不需要首部长度字段。接收端先读取 8 字节报头，再结合 UDP 长度确定完整数据报。

2.2.3 UDP 与 TCP 的基本定位

对比项	UDP	TCP
连接方式	无连接	面向连接
数据形式	面向数据报	面向字节流
消息边界	数据报天然保留	应用自行划分
可靠性	协议不保证	ACK、排序、重传

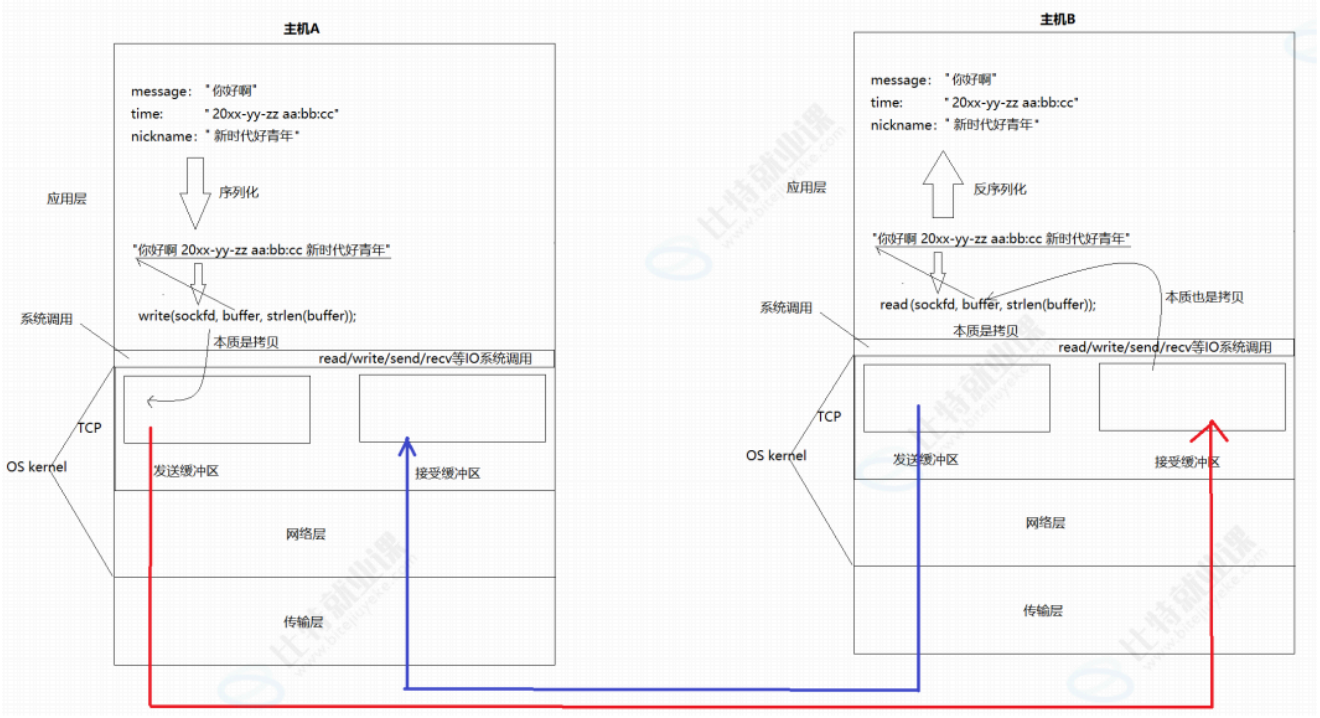
对比项	UDP	TCP
控制开销	固定 8 字节报头	20 到 60 字节报头、连接状态
典型场景	实时媒体、短请求、广播	文件、网页、登录、命令

UDP 也可以在应用层增加确认、序号和重传，但这种“选择性可靠性”应当按业务需求设计，而不是机械复制全部 TCP 机制。

2.3 TCP 面向字节流

2.3.1 应用缓冲区与内核缓冲区

TCP socket 在内核中关联一个发送缓冲区和一个接收缓冲区。应用调用 `write/send` 时，数据先从用户空间复制到 TCP 发送缓冲区；接收端把到达的数据放入 TCP 接收缓冲区，应用调用 `read/recv` 时再复制到用户空间。




```
应用消息
  -> 序列化
  -> write 复制到发送缓冲区
  -> TCP 分段发送
  -> 对端 TCP 接收缓冲区
  -> read 复制到应用
  -> 反序列化
```

- `write` 返回成功通常表示数据已经交给内核缓冲区；
- TCP 的 ACK 表示对端 TCP 层收到数据；
- ACK 不表示对端应用已经读取或业务处理成功；
- 应用读取后，接收缓冲区空间才会真正释放。

2.3.2 TCP 不保存应用消息边界

TCP 把多次写入视为一条连续字节流。发送次数与接收次数没有固定对应关系：

```
发送：write("hello") + write("world")
接收：可能一次得到 "helloworld"
      也可能先得到 "hel"，再得到 "loworld"
```

这是字节流语义，不是数据损坏。若应用需要区分完整消息，就必须在应用层定义边界。

2.4 应用层如何划分消息

2.4.1 TCP 粘包的本质

所谓 TCP“粘包”不是 TCP 把两个业务包错误拼接，而是应用无法从连续字节流判断一条业务消息在哪里结束。

一次读取可能出现：

- 只读到半条消息；
- 正好读到一条消息；
- 同时读到多条消息。

应用必须缓存不完整数据，直到能解析出完整消息。

2.4.2 常见消息边界方案

方案	适用情况	注意点
固定长度	所有消息等长	不足时需要填充，可能浪费空间
固定头部 + 长度字段	二进制协议	需要处理头部不完整和长度上限
特殊分隔符	文本协议	必须处理转义和正文冲突

方案	适用情况	注意点
序列化协议	字段复杂、跨语言	一般仍依赖长度或类型信息

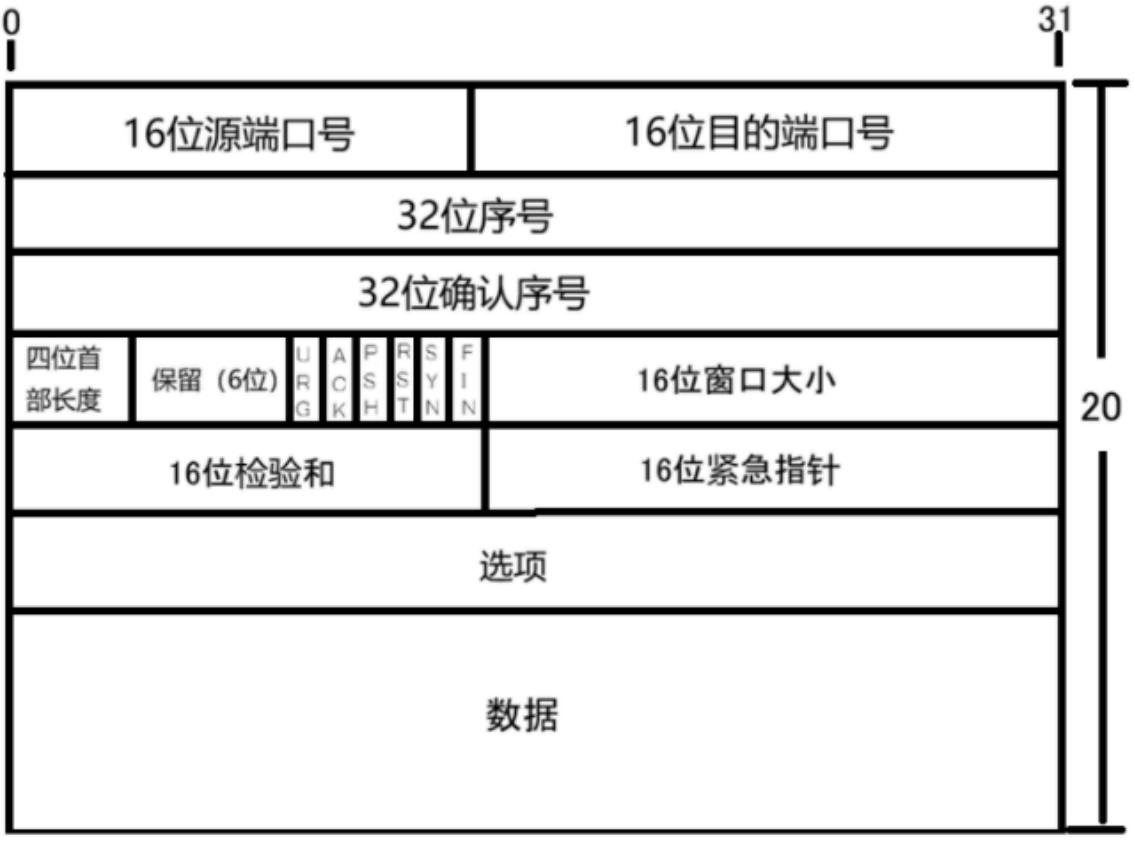
TCP 不提供类似 UDP 的“消息长度”字段；它只有字节序号。序列化解决的是消息内部字段结构，消息边界仍需要协议设计明确。

3. TCP 报文格式

3.1 基础首部

3.1.1 20 到 60 字节

TCP 基础首部为 20 字节，选项最多 40 字节，所以总首部长度是 20 到 60 字节。



四位首部长度字段以 4 字节为单位：

```
TCP 首部长度 = 首部长度字段值 * 4
```

字段值	实际首部长度
5	20 字节
10	40 字节
15	60 字节

普通数据报文经常不带选项；建立连接时可能携带 MSS、窗口扩大因子、时间戳等选项，因此不能始终假设数据从第 21 字节开始。

3.1.2 端口、序号与确认序号

字段	长度	作用
源端口	16 位	标识发送进程
目的端口	16 位	标识接收进程并完成分用
序号	32 位	标识本段数据在字节流中的位置
确认序号	32 位	表示下一个期望收到的字节序号

TCP 序号按字节编号，而不是按报文编号。一个数据段携带 `L` 字节，首字节序号为 `seq` 时，下一段通常从 `seq + L` 开始。

确认号采用累计语义：`ack=2001` 表示 2001 之前的字节都已经按序收到。更详细的确认和重传机制见 2026-9-18 Linux网络--TCP可靠传输与流量控制。

3.1.3 窗口、校验和与选项

16 位窗口字段表示发送该报文的端点还剩多少接收空间。双方填写的都是自己的接收能力。

TCP 校验和覆盖 TCP 报头和有效载荷。接收端校验失败时会丢弃该报文并等待重传。TCP 使用的是 TCP 校验和算法，不是 CRC。

常见选项包括：

- MSS：限制 TCP 有效载荷最大长度；
- 窗口扩大因子：扩大接收窗口的可表示范围；
- 时间戳：支持 RTT 测量和防护旧报文；

- SACK：允许选择性确认部分区间。

3.2 标志位

3.2.1 六个常用标志位

标志位	作用
ACK	确认序号有效
SYN	请求或确认建立连接
FIN	发送方不再发送普通数据
RST	立即复位连接
PSH	提示接收端应用尽快读取数据
URG	紧急指针有效

ACK、SYN、FIN 和 RST 的连接状态含义见连接管理专题。这里重点区分它们所表达的控制类型。

3.2.2 PSH 与 URG

PSH 不是“窗口已满”的标志，而是催促接收端操作系统尽快把数据交给应用。即使发送方设置了 PSH，应用不调用 `read`，接收缓冲区也不会自动释放。

URG 是紧急指针字段的开关。URG 为 1 时，紧急指针指向字节流中需要优先处理的位置。接收端可以通过带外数据接口读取：

```
recv(sockfd, &ch, 1, MSG_OOB);
```

URG 适合少量紧急控制信息，不适合承载大量业务数据。控制信息较多时，独立控制连接通常更清晰。

3.2.3 ACK 标志与捎带应答

ACK 为 1 时，确认序号字段才有效。普通数据报文也可以带 ACK，因为它能在发送自身数据的同时确认对端数据。

一个 TCP 报文可以同时包含：

```
seq     = 本端发送数据的序号
ack_seq = 已按序收到对端数据的边界
ACK     = 1
payload = 本端业务数据
```

这就是捎带应答。它属于传输效率优化，机制细节见 2026-9-18 Linux网络--TCP拥塞控制与传输优化。

4. TCP 之下的 IP 承载

4.1 封装关系

TCP 段不会直接进入网络，而要向下交给 IP：

```
应用数据
  -> TCP 段
  -> IP 报文
  -> 链路层帧
  -> 物理网络
```

IP 负责跨多个子网和路由器把主机到主机的报文送到目的主机，提供尽力而为的转发。TCP 在这个可能丢包、乱序的通道上提供端到端可靠字节流。

4.2 协议号与向上分用

IP 报头中的 8 位协议字段表示有效载荷属于哪个传输层协议：

- 协议号 6：TCP；
- 协议号 17：UDP。

接收端先根据目的 IP 确认目标主机，再由 IP 协议字段交给 TCP/UDP，最后由目的端口把数据交给对应进程。

5. 本章边界与后续阅读

本篇建立了后续四篇共同使用的概念：

```
TCP = 连续字节流
    + 字节序号
    + 应用层自定义消息边界
    + 端口分用
    + 带控制字段的 TCP 报头
    + IP 的主机到主机承载
```

本篇不展开确认重传和滑动窗口，见 2026-9-18 Linux网络--TCP可靠传输与流量控制；拥塞控制与延迟应答见 2026-9-18 Linux网络--TCP拥塞控制与传输优化；握手、挥手和状态诊断见 2026-9-18 Linux网络--TCP连接管理与异常诊断；socket API 与部署见 2026-9-18 Linux网络--TCP Socket编程与工程实践。